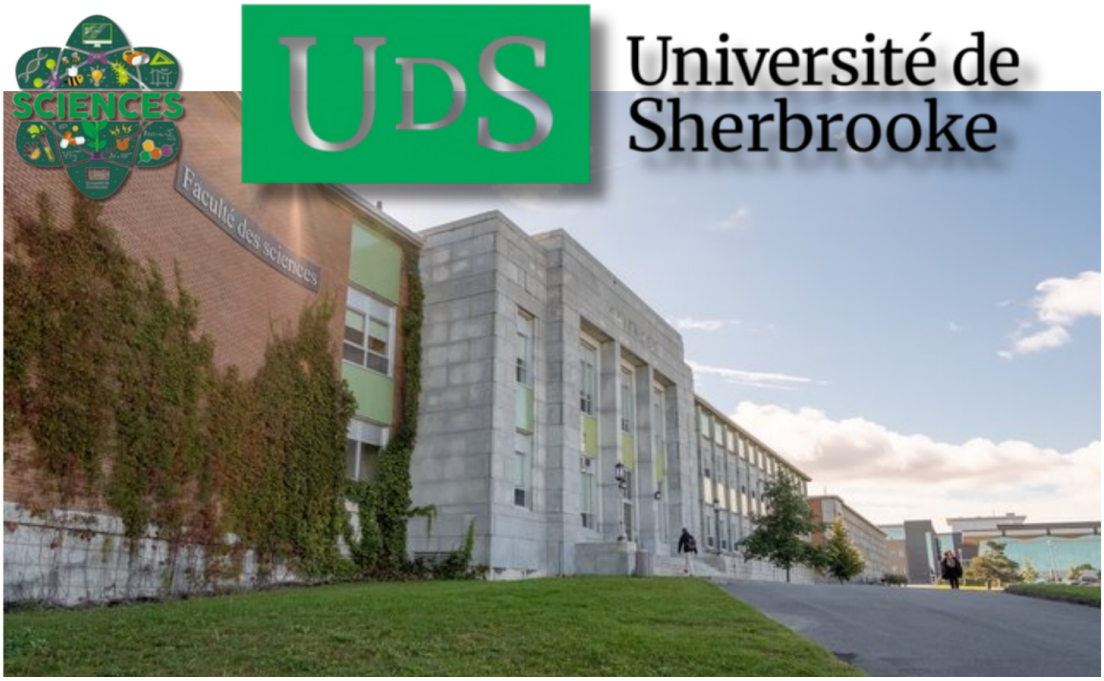


Devoir 2

IFT814-18-51 - Cryptographie

Automne 2026

Professeur : Martin FISET

Groupe 10 :

Cheikh Ahmadou Tidiane Ly → **lyzc3821**
Blakoua Adja → **adjb1401**
Odounlami Lucien Gbessemehlan → **gbeo6367**
Rodier Marcel Tumamo Simo → **tumr4770**

Déclaration d'usage des outils d'IA générative

Niveau 1 des balises de la Faculté des sciences. Un outil d'IA générative a été utilisé de manière limitée pour **se faire expliquer des notions** (logarithme discret, OAEP, PSS, principe de Miller-Rabin) et pour **aider à structurer la rédaction**. Les algorithmes demandés (exponentiation rapide, Euclide étendu, Miller-Rabin, Diffie-Hellman, RSA, signatures) ont été écrits et compris par l'équipe ; toutes les traces proviennent de l'exécution de notre propre code et ont été comparées aux valeurs `[Verification]` de la fiche. Aucun code, trace ou réponse n'a été repris tel quel sans vérification.

Table des matières

Déclaration d'usage des outils d'IA générative **1**

Conventions et remarques générales **3**

1 Partie 1 – Diffie-Hellman modulo $p = 2^{61} - 1$ **3**

1.1 Implémentation (1.1), trace imposée (1.2), chiffrement par flot (1.3) 3

1.2 1.4 – Espionnage passif : ce qu'Ève peut et ne peut pas faire 6

1.3 1.5 – Attaque active : l'homme du milieu 7

2 Partie 2 – RSA « textbook » **10**

2.1 2.1 – Génération de clés : Miller-Rabin et Euclide étendu 10

2.2 2.2 – Chiffrement et déchiffrement 10

2.3 2.3 – Cas limites et déterminisme 10

2.4 2.4 – Malléabilité 10

2.5 2.5 – Attaque par dictionnaire 11

3 Partie 3 – Signatures RSA « textbook » **15**

3.1 3.1 – Implémentation 15

3.2 3.2 – Oracle de signature simulé 15

3.3 3.3 – Forgerie existentielle 16

3.4 3.4 – Forgerie par message choisi (multiplicative) 16

4 Discussion : limites et améliorations **19**

5 Bibliographie et sources consultées **20**

Conventions et remarques générales

Toutes les primitives sont **implémentées à la main** : l'exponentiation modulaire $a^b \bmod n$ est calculée par *carré-et-multiplication* ($\mathcal{O}(\log b)$ multiplications), l'inverse modulaire par l'*algorithme d'Euclide étendu*, et la primalité par *Miller-Rabin*. Aucune bibliothèque de chiffrement n'est utilisée. Le seul aléa provient du module `secrets` (générateur cryptographiquement sûr). $\oplus$ désigne le OU exclusif bit à bit.

Chaque script affiche ses résultats à côté de la valeur `[Verification x.y]` attendue de la fiche **Équipe 05**. **Les 16 vérifications de la fiche sont reproduites exactement.**

1 Partie 1 – Diffie-Hellman modulo $p = 2^{61} - 1$

On travaille dans $\mathbb{Z}_p^*$ avec $p = 2^{61} - 1 = 2305843009213693951$ (premier de Mersenne) et $g = 37$ (racine primitive, d'ordre $p - 1$).

1.1 Implémentation (1.1), trace imposée (1.2), chiffrement par flot (1.3)

Protocole. `Gen` tire un exposant privé aléatoire dans $\{2, \ldots, p-2\}$; `PubKey`$(x) = g^x \bmod p$; `Secret` calcule (clé publique reçue)$^{\text{clé privée}} \bmod p$. Comme $(g^x)^y = (g^y)^x = g^{xy}$, Alice et Bob obtiennent le même secret.

1.3 – chiffrement par flot. Le secret K est tronqué à 32 bits, soit 4 octets en représentation *big-endian*, et sert de clé : $c[i] = m[i] \oplus k[i \bmod 4]$ sur l'encodage ASCII du message. Le XOR étant involutif, le déchiffrement réutilise la même clé.

Listing 1 – partie1_diffie_hellman.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
IFT814 - Devoir 2 - Equipe 05
Partie 1 : Diffie-Hellman modulo p = 2^61 - 1 (nombre premier de Mersenne)

Couvre :
  1.1 Implementation du protocole (Gen, PubKey, Secret) + exemple complet
  1.2 Trace avec les cles imposees de la fiche
  1.3 Chiffrement par flot XOR avec la cle derivee (K tronque a 32 bits)

Aucune bibliotheque de chiffrement n'est utilisee : l'exponentiation modulaire
est implementee a la main par "carre-et-multiplication". Le seul tirage
aleatoire (1.1) utilise le module `secrets` (generateur cryptographiquement sur).
"""

import secrets

# ---------------------------------------------------------------------------
# Parametres publics (identiques pour toute la classe, voir la fiche)
# ---------------------------------------------------------------------------
P = 2**61 - 1              # = 2305843009213693951, premier de Mersenne
G = 37                     # racine primitive modulo P (ordre multiplicatif = P-1)


# ---------------------------------------------------------------------------
# Exponentiation modulaire rapide : a^b mod n (carre-et-multiplication)
# Complexite : O(log b) multiplications, au lieu de O(b) pour la methode naive.
# ---------------------------------------------------------------------------
def pow_mod(a, b, n):
    """Retourne a^b mod n sans jamais manipuler a^b en entier."""
    resultat = 1
```

```
    base = a % n
    exposant = b
    while exposant > 0:
        if exposant & 1:            # bit de poids faible a 1 -> on multiplie
            resultat = (resultat * base) % n
        base = (base * base) % n     # on met la base au carre a chaque etape
        exposant >>= 1               # on passe au bit suivant
    return resultat


# ------------------------------------------------------------------------
# 1.1 Le protocole Diffie-Hellman
# ------------------------------------------------------------------------
def gen(p=P):
    """Choisit un exposant prive aleatoire dans {2, ..., p-2} (generateur sur)."""
    # secrets.randbelow(n) renvoie un entier uniforme dans [0, n-1].
    # On veut [2, p-2] : on tire dans [0, p-4] puis on decale de 2.
    return 2 + secrets.randbelow(p - 3)


def pub_key(x, g=G, p=P):
    """Demi-cle publique : h = g^x mod p."""
    return pow_mod(g, x, p)


def secret(cle_privee, cle_publique_recue, p=P):
    """Secret partage : (cle_publique_recue)^(cle_privee) mod p."""
    return pow_mod(cle_publique_recue, cle_privee, p)


# ------------------------------------------------------------------------
# 1.3 Chiffrement par flot XOR avec la cle derivee
# ------------------------------------------------------------------------
def cle_flot_depuis_secret(K, nb_octets=4):
    """Tronque le secret K a 32 bits et le represente sur 4 octets big-endian."""
    K32 = K % (2**32)                       # troncature a 32 bits
    return K32.to_bytes(nb_octets, 'big')   # representation "big-endian"/boutisme


def chiffrer_xor(message_bytes, cle_bytes):
    """c[i] = m[i] XOR cle[i mod len(cle)], octet par octet."""
    k = len(cle_bytes)
    return bytes(message_bytes[i] ^ cle_bytes[i % k] for i in range(len(message_bytes)))


# Le XOR est involutif : dechiffrer = rechiffrer avec la meme cle.
def dechiffrer_xor(cipher_bytes, cle_bytes):
    return chiffrer_xor(cipher_bytes, cle_bytes)


# ========================================================================
# Demonstrations
# ========================================================================
def demo_1_1():
    print("=" * 70)
    print("1.1  Exemple d'execution complet (cles aleatoires choisies par le code)")
    print("=" * 70)
    # Alice et Bob tirent chacun une cle privee aleatoire.
    a = gen()
    b = gen()
    hA = pub_key(a)          # Alice publie h_A = g^a
    hB = pub_key(b)          # Bob   publie h_B = g^b
    # Chacun calcule le secret a partir de la demi-cle recue.
    K_alice = secret(a, hB)  # (h_B)^a
    K_bob   = secret(b, hA)  # (h_A)^b
```

```
    print(f"  cle privee Alice a = {a}")
    print(f"  cle privee Bob   b = {b}")
    print(f"  h_A = g^a mod p = {hA}")
    print(f"  h_B = g^b mod p = {hB}")
    print(f"  secret cote Alice = {K_alice}")
    print(f"  secret cote Bob   = {K_bob}")
    print(f"  --> les deux secrets coincident ? {K_alice == K_bob}")
    print()


def demo_1_2():
    print("=" * 70)
    print("1.2  Trace avec les cles imposees de la fiche (Equipe 05)")
    print("=" * 70)
    x = 974264167110881       # cle privee imposee d'Alice
    y = 23179607248816        # cle privee imposee de Bob
    hA = pub_key(x)
    hB = pub_key(y)
    K = secret(x, hB)         # = h_B^x
    K_bis = secret(y, hA)     # = h_A^y (doit etre egal)
    print(f"  x = {x}")
    print(f"  y = {y}")
    print(f"  h_A = g^x mod p = {hA}")
    print(f"      [Verification 1.2 attendue] 1728559941401253307")
    print(f"  h_B = g^y mod p = {hB}")
    print(f"      [Verification 1.2 attendue] 1512476708919266528")
    print(f"  K = h_B^x = h_A^y mod p = {K}  (coherence : {K == K_bis})")
    print(f"  K mod 2^32 = {K % (2**32)}")
    print(f"      [Verification 1.2 attendue] 3512349958")
    print()
    return K


def demo_1_3(K):
    print("=" * 70)
    print("1.3  Chiffrement par flot XOR du message de la fiche")
    print("=" * 70)
    message = "nouvelle cle generee aujourd hui"
    m_bytes = message.encode('ascii')
    cle = cle_flot_depuis_secret(K)       # K tronque a 32 bits -> 4 octets big-endian
    c_bytes = chiffrer_xor(m_bytes, cle)
    print(f"  message clair        : {message!r}")
    print(f"  cle (K mod 2^32) hex : {cle.hex()}")
    print(f"  chiffre (hex)        : {c_bytes.hex()}")
    print(f"      [Verification 1.3 attendue]"
          f" bf354070b4365963f1395963f13d5068b4285063f13b406cbe2f4762f132406f")
    # Verification du dechiffrement (doit redonner le clair)
    clair = dechiffrer_xor(c_bytes, cle).decode('ascii')
    print(f"  dechiffre            : {clair!r}  (identique : {clair == message})")
    print()


if __name__ == "__main__":
    print(f"p = 2^61 - 1 = {P}")
    print(f"g = {G}\n")
    demo_1_1()
    K = demo_1_2()
    demo_1_3(K)
```

Exécution. (Les lignes « attendue » rappellent la fiche ; elles coïncident.)

```
p = 2^61 - 1 = 2305843009213693951
g = 37

======================================================================
1.1  Exemple d'execution complet (cles aleatoires choisies par le code)
```

```
======================================================================
  cle privee Alice a = 545006843325181653
  cle privee Bob   b = 1669349178431416828
  h_A = g^a mod p = 631299493525031151
  h_B = g^b mod p = 480137258481510504
  secret cote Alice = 140403880639568327
  secret cote Bob   = 140403880639568327
  --> les deux secrets coincident ? True

======================================================================
1.2  Trace avec les cles imposees de la fiche (Equipe 05)
======================================================================
  x = 974264167110881
  y = 23179607248816
  h_A = g^x mod p = 1728559941401253307
      [Verification 1.2 attendue] 1728559941401253307
  h_B = g^y mod p = 1512476708919266528
      [Verification 1.2 attendue] 1512476708919266528
  K = h_B^x = h_A^y mod p = 397785387543180550  (coherence : True)
  K mod 2^32 = 3512349958
      [Verification 1.2 attendue] 3512349958

======================================================================
1.3  Chiffrement par flot XOR du message de la fiche
======================================================================
  message clair        : 'nouvelle cle generee aujourd hui'
  cle (K mod 2^32) hex : d15a3506
  chiffre (hex)        : bf354070b4365963f1395963f13d5068b4285063f13b406cbe2f4762f132406f
      [Verification 1.3 attendue] bf354070b4365963f1395963f13d5068b4285063f13b406cbe2f4762f132406f
  dechiffre            : 'nouvelle cle generee aujourd hui'  (identique : True)
```

1.2 1.4 – Espionnage passif : ce qu'Ève peut et ne peut pas faire

Problème à résoudre. Ève voit $p, g, h_A = g^x, h_B = g^y$ mais ni x ni y. Pour obtenir $K = g^{xy}$, il lui suffirait de retrouver un seul exposant, par exemple x tel que $g^x \equiv h_A \pmod{p}$: c'est le **problème du logarithme discret** (DLP). L'exponentiation est facile, mais son inverse n'a pas d'algorithme efficace connu dans un groupe bien choisi (hypothèse Diffie-Hellman calculatoire).

Pourquoi c'est « difficile »… mais pas ici. Les meilleurs algorithmes génériques (pas-de-bébé/pas-de-géant, ρ de Pollard) coûtent $\approx \sqrt{p}$ opérations. Avec $p = 2^{61} - 1$:

$$\sqrt{p} \approx 2^{30.5} \approx 1{,}5 \times 10^9 \text{ opérations,}$$

soit **quelques secondes à quelques minutes** sur une machine ordinaire ($\sim 10^9$ mult./s). 61 bits n'offrent donc *aucune* sécurité réelle.

Taille de p	Coût de l'attaque	Statut
61 bits (ce TP)	$\approx 2^{30} \approx 10^9$ op. $\rightarrow$ secondes	cassé en direct
2048 bits (min.)	$\approx 2^{112}$ op.	hors de portée
3072 bits (recomm.)	$\approx 2^{128}$ op.	hors de portée

Pour un grand p, le *crible de corps de nombres* (sous-exponentiel) fait qu'il faut 2048–3072 bits pour n'atteindre que ∼112–128 bits de sécurité.

Ève peut-elle déchiffrer 1.3 sans K ? Oui, sans toucher au logarithme discret : la clé de flot ne fait que 32 bits, *répétée* tous les 4 octets. (1) Recherche exhaustive sur $2^{32} \approx 4 \times 10^9$ clés, en retenant celle qui donne un *français lisible* (le texte clair attendu sert de test). (2) Encore plus simple, attaque à clair connu : 4 octets de clair devinés suffisent à récupérer les 4 octets de clé par clé = clair $\oplus$ chiffré, puis à tout déchiffrer. La vraie faiblesse est donc la **dérivation de clé** (clé courte et périodique), pas seulement la taille de p.

1.3 1.5 – Attaque active : l'homme du milieu

Étapes. (1) Mallory intercepte h_A et h_B et ne les relaie pas. (2) Il envoie sa propre demi-clé g^{m_A} à Alice et g^{m_B} à Bob. (3) Alice calcule $K_A = (g^{m_A})^x$ et Mallory le même $K_A = (h_A)^{m_A}$; symétriquement Bob et Mallory partagent K_B. (4) Mallory déchiffre avec K_A, lit, modifie, re-chiffre avec K_B et transmet à Bob, qui ne détecte rien.

Nombre de secrets. Il existe **deux** secrets distincts : K_A (Alice↔Mallory) et K_B (Bob↔Mallory). Aucun secret commun direct entre Alice et Bob, qui croient pourtant n'en partager qu'un.

Ce que Mallory peut faire sur les messages de 1.3 : les lire, les modifier, les supprimer, en injecter – il connaît les deux clés.

Propriété manquante. L'**authentification des demi-clés** (origine des entités) : on signe les demi-clés publiques (Diffie-Hellman authentifié / station-to-station) ou on les lie à un **certificat** (PKI). Ne pouvant forger ces signatures, Mallory ne peut plus substituer ses clés sans être détecté.

Listing 2 – partie1_mitm.py

```python
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
IFT814 - Devoir 2 - Equipe 05
Partie 1.5 : Attaque de l'homme du milieu (Man-In-The-Middle) sur Diffie-Hellman
             NON authentifie.

Trois entites logicielles (Alice, Bob, Mallory). Mallory controle le canal :
il intercepte les deux demi-cles publiques et negocie SEPAREMENT un secret
avec Alice (K_A) et un autre avec Bob (K_B). Il peut donc dechiffrer, lire,
modifier, puis re-chiffrer tout message qui transite, sans qu'Alice ni Bob
ne s'en apercoive.

On reutilise le chiffrement par flot XOR de la question 1.3.
"""

import secrets

P = 2**61 - 1
G = 37


def pow_mod(a, b, n):
    """a^b mod n par carre-et-multiplication."""
    r, base, e = 1, a % n, b
    while e > 0:
        if e & 1:
            r = (r * base) % n
        base = (base * base) % n
        e >>= 1
    return r


def gen():
    return 2 + secrets.randbelow(P - 3)


def cle_flot(K):
    """K tronque a 32 bits -> 4 octets big-endian (comme en 1.3)."""
    return (K % (2**32)).to_bytes(4, 'big')


def xor(data, cle):
    return bytes(data[i] ^ cle[i % len(cle)] for i in range(len(data)))


```

```python
# ------------------------------------------------------------------------
# Les trois entites
# ------------------------------------------------------------------------
class Partie:
    """Alice ou Bob : possede une cle privee et derive un secret."""
    def __init__(self, nom):
        self.nom = nom
        self.prive = gen()
        self.publique = pow_mod(G, self.prive, P)   # g^prive mod p
        self.K = None

    def etablir_secret(self, demi_cle_recue):
        self.K = pow_mod(demi_cle_recue, self.prive, P)
        return self.K


class Mallory:
    """L'attaquant au milieu : deux cles privees, une par victime."""
    def __init__(self):
        # Une cle privee pour parler a Alice, une autre pour parler a Bob.
        self.prive_vers_alice = gen()
        self.prive_vers_bob = gen()
        self.pub_vers_alice = pow_mod(G, self.prive_vers_alice, P)
        self.pub_vers_bob = pow_mod(G, self.prive_vers_bob, P)
        self.K_avec_alice = None   # secret partage Mallory <-> Alice
        self.K_avec_bob = None     # secret partage Mallory <-> Bob


def simulation_mitm():
    print("=" * 70)
    print("1.5  Simulation d'une attaque de l'homme du milieu")
    print("=" * 70)

    alice = Partie("Alice")
    bob = Partie("Bob")
    mallory = Mallory()

    # --- Echange des demi-cles, TOUT passe par Mallory ---------------------
    # Alice croit envoyer h_A a Bob ; Mallory l'intercepte et renvoie SA
    # propre demi-cle a Bob. Symetriquement pour Bob.
    print("\n[Echange des demi-cles -- le canal est controle par Mallory]")
    print(f"  Alice envoie h_A = {alice.publique}")
    print(f"  Bob   envoie h_B = {bob.publique}")
    print(f"  Mallory remplace : Alice recoit {mallory.pub_vers_alice} (de Mallory)")
    print(f"  Mallory remplace : Bob   recoit {mallory.pub_vers_bob} (de Mallory)")

    # Alice derive son secret avec ce qu'elle a recu (= demi-cle de Mallory).
    K_A = alice.etablir_secret(mallory.pub_vers_alice)
    # Mallory derive le MEME secret de son cote (avec la vraie demi-cle d'Alice).
    mallory.K_avec_alice = pow_mod(alice.publique, mallory.prive_vers_alice, P)

    # Idem cote Bob.
    K_B = bob.etablir_secret(mallory.pub_vers_bob)
    mallory.K_avec_bob = pow_mod(bob.publique, mallory.prive_vers_bob, P)

    print("\n[Secrets etablis]")
    print(f"  Alice  croit partager  K_A = {K_A}")
    print(f"  Mallory partage avec Alice = {mallory.K_avec_alice}"
          f"  (identique : {K_A == mallory.K_avec_alice})")
    print(f"  Bob    croit partager  K_B = {K_B}")
    print(f"  Mallory partage avec Bob   = {mallory.K_avec_bob}"
          f"  (identique : {K_B == mallory.K_avec_bob})")
    print(f"\n  --> Il existe DEUX secrets distincts (K_A != K_B : {K_A != K_B}).")
    print(f"      Alice et Bob croient pourtant partager une seule cle commune.")

```

```
    # --- Un message transite : Alice -> (Mallory) -> Bob ------------------
    print("\n[Alice envoie un message qu'elle croit destine directement a Bob]")
    message = "transfere 100 dollars a Bob"
    print(f"  message clair d'Alice : {message!r}")

    # Alice chiffre avec K_A (qu'elle croit partage avec Bob).
    c_alice = xor(message.encode('ascii'), cle_flot(K_A))
    print(f"  chiffre d'Alice (hex) : {c_alice.hex()}")

    # Mallory intercepte : il dechiffre avec K_A (qu'il connait).
    lu_par_mallory = xor(c_alice, cle_flot(mallory.K_avec_alice)).decode('ascii')
    print(f"\n[Mallory intercepte et DECHIFFRE avec K_A]")
    print(f"  Mallory lit : {lu_par_mallory!r}")
    assert lu_par_mallory == message, "Mallory devrait lire le clair exact"

    # Mallory MODIFIE le message, puis le re-chiffre avec K_B pour Bob.
    message_modifie = "transfere 900 dollars a Mallory"
    c_pour_bob = xor(message_modifie.encode('ascii'), cle_flot(mallory.K_avec_bob))
    print(f"  Mallory reecrit : {message_modifie!r}")
    print(f"  re-chiffre pour Bob (hex) : {c_pour_bob.hex()}")

    # Bob dechiffre avec K_B : il obtient le message falsifie, sans rien detecter.
    recu_par_bob = xor(c_pour_bob, cle_flot(K_B)).decode('ascii')
    print(f"\n[Bob dechiffre avec K_B]")
    print(f"  Bob lit : {recu_par_bob!r}")
    print(f"  Bob a-t-il recu le message original d'Alice ? "
          f"{recu_par_bob == message}")
    print(f"  Bob detecte-t-il une alteration ? Non : le dechiffrement est")
    print(f"  parfaitement valide de son point de vue.")
    print("\n[Conclusion] Mallory a lu ET modifie le message en restant invisible.")


if __name__ == "__main__":
    simulation_mitm()
```

```
======================================================================
1.5  Simulation d'une attaque de l'homme du milieu
======================================================================

[Echange des demi-cles -- le canal est controle par Mallory]
  Alice envoie h_A = 1720790412519098298
  Bob   envoie h_B = 1504760924712126204
  Mallory remplace : Alice recoit 2223127521876272003 (de Mallory)
  Mallory remplace : Bob   recoit 1386214851722151174 (de Mallory)

[Secrets etablis]
  Alice  croit partager  K_A = 165641871430827460
  Mallory partage avec Alice = 165641871430827460  (identique : True)
  Bob    croit partager  K_B = 605989595325168063
  Mallory partage avec Bob   = 605989595325168063  (identique : True)

  --> Il existe DEUX secrets distincts (K_A != K_B : True).
      Alice et Bob croient pourtant partager une seule cle commune.

[Alice envoie un message qu'elle croit destine directement a Bob]
  message clair d'Alice : 'transfere 100 dollars a Bob'
  chiffre d'Alice (hex) : fee2dcaaf9f6d8b6efb08cf4bab0d9abe6fcdcb6f9b0dce4c8ffdf

[Mallory intercepte et DECHIFFRE avec K_A]
  Mallory lit : 'transfere 100 dollars a Bob'
  Mallory reecrit : 'transfere 900 dollars a Mallory'
  re-chiffre pour Bob (hex) : f28218d1f5961ccde3d0408fb6d01dd0ea9c18cdf5d0189fcb9115d3e98200

[Bob dechiffre avec K_B]
  Bob lit : 'transfere 900 dollars a Mallory'
  Bob a-t-il recu le message original d'Alice ? False
  Bob detecte-t-il une alteration ? Non : le dechiffrement est
  parfaitement valide de son point de vue.

[Conclusion] Mallory a lu ET modifie le message en restant invisible.
```

2 Partie 2 – RSA « textbook »

Fiche : $p = 209287069$, $q = 221536921$, $e = 65537$.

2.1 2.1 – Génération de clés : Miller-Rabin et Euclide étendu

Miller-Rabin (principe). Pour tester n, on écrit $n-1 = 2^s d$ (d impair) et, pour plusieurs témoins aléatoires a, on vérifie que la suite $a^d, a^{2d}, \ldots \bmod n$ atteint 1 *uniquement en passant par* -1 (car modulo un premier, les seules racines carrées de 1 sont ± 1) ; une seule violation prouve que n est composé, et après k témoins la probabilité d'erreur est $\leq 4^{-k}$. Ici p et q sont déclarés premiers.

N **et** $\varphi(N)$**.** $N = pq = 46364812871374549$ et $\varphi(N) = (p-1)(q-1) = 46364812440550560$.

Calcul de $d = e^{-1} \bmod \varphi(N)$ **par Euclide étendu.** L'algorithme donne $\gcd(e, \varphi) = 1$ (donc e inversible) et le coefficient de Bézout u tel que $eu + \varphi v = 1$, d'où $d = u \bmod \varphi$. Trace (quotient, reste, coefficient u de e) :

```
etape |   quotient |               reste |        u (coef de e)
  0   |     -      |               65537 |                   1
  1   |     -      |    46364812440550560 |                   0
  2   |     0      |               65537 |                   1
  3   | 707460097968 |              21744 |        -707460097968
  4   |     3      |                 305 |        2122380293905
  5   |    71      |                  89 |     -151396460965223
  6   |     3      |                  38 |      456311763189574
  7   |     2      |                  13 |    -1064019987344371
  8   |     2      |                  12 |     2584351737878316
  9   |     1      |                   1 |    -3648371725222687
-----------------------------------------------------------------------
gcd(e,phi)=1 ; u = -3648371725222687
d = u mod phi(N) = 42716440715327873   [Verification 2.1 : OK]
```

2.2 2.2 – Chiffrement et déchiffrement

$c = m^e \bmod N$ et $m' = c'^d \bmod N$. Les deux valeurs correspondent à `[Verification 2.2]` (voir trace plus bas).

2.3 2.3 – Cas limites et déterminisme

$\mathrm{Enc}(0) = 0$, $\mathrm{Enc}(1) = 1$, $\mathrm{Enc}(N-1) = N-1$: ce sont des **points fixes** ($N - 1 \equiv -1$ et e impair). Un attaquant *reconnaît* ces clairs sans aucune cryptanalyse. De plus RSA textbook est **déterministe** : deux cryptogrammes identiques trahissent deux clairs identiques (fuite directe, pas d'indistinguabilité IND-CPA). C'est ce que corrige **RSA-OAEP** (chiffrement probabiliste).

2.4 2.4 – Malléabilité

RSA textbook est homomorphe : $\mathrm{Enc}(m_1)\mathrm{Enc}(m_2) = \mathrm{Enc}(m_1 m_2 \bmod N)$. Donc

$$c_{\mathrm{mal}} = c \cdot 2^e \bmod N = \mathrm{Enc}(m)\,\mathrm{Enc}(2) = \mathrm{Enc}(2m \bmod N),$$

sans connaître m ni d. On vérifie $\mathrm{Dec}(c_{\mathrm{mal}}) = 2m \bmod N$ (`[Verification 2.4]`). **Scénario réel** : un service de facturation qui chiffre un montant en RSA textbook peut voir son montant *doublé* (ou multiplié par k via $c \cdot k^e$) par un attaquant qui ne connaît ni le clair ni la clé. Parade : **RSA-OAEP**, dont le padding rend tout cryptogramme trafiqué invalide.

2.5 2.5 – Attaque par dictionnaire

RSA textbook étant déterministe et e, N publics, un attaquant qui sait que m appartient à un petit ensemble *chiffre chaque candidat* et compare à c. Sur un dictionnaire de 1000 candidats contenant la vraie valeur, m est retrouvé en 500 essais. **RSA-OAEP** (probabiliste) rend l'attaque impraticable : le même m produit un c différent et imprévisible à chaque chiffrement.

Listing 3 – partie2_rsa.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
IFT814 - Devoir 2 - Equipe 05
Partie 2 : RSA "textbook" (sans padding)

Couvre :
  2.1 Generation de la paire de cles (test de primalite Miller-Rabin,
      calcul de d = e^-1 mod phi(N) par l'algorithme d'Euclide etendu)
  2.2 Chiffrement et dechiffrement
  2.3 Cas limites : Enc(0), Enc(1), Enc(N-1), determinisme
  2.4 Malleabilite (homomorphisme multiplicatif)
  2.5 Attaque par dictionnaire sur un petit espace de messages

Toutes les primitives (exponentiation, primalite, inverse modulaire) sont
implementees a la main. Le seul aleatoire (temoins de Miller-Rabin) utilise
le module `secrets`.
"""

import secrets

# ---------------------------------------------------------------------------
# Parametres imposes par la fiche (Equipe 05)
# ---------------------------------------------------------------------------
P = 209287069
Q = 221536921
E = 65537


# ---------------------------------------------------------------------------
# Exponentiation modulaire rapide
# ---------------------------------------------------------------------------
def pow_mod(a, b, n):
    r, base, e = 1, a % n, b
    while e > 0:
        if e & 1:
            r = (r * base) % n
        base = (base * base) % n
        e >>= 1
    return r


# ---------------------------------------------------------------------------
# Algorithme d'Euclide etendu : renvoie (g, u, v) tel que a*u + b*v = g = pgcd(a,b)
# ---------------------------------------------------------------------------
def euclide_etendu(a, b):
    # Version iterative, en gardant la trace des coefficients de Bezout.
    u_prec, u = 1, 0
    v_prec, v = 0, 1
    r_prec, r = a, b
    while r != 0:
        quotient = r_prec // r
        r_prec, r = r, r_prec - quotient * r
        u_prec, u = u, u_prec - quotient * u
        v_prec, v = v, v_prec - quotient * v
    # a*u_prec + b*v_prec = r_prec = pgcd(a, b)
```

```
    return r_prec, u_prec, v_prec


def inverse_mod(a, n):
    """Inverse de a modulo n (necessite pgcd(a, n) = 1)."""
    g, u, _ = euclide_etendu(a % n, n)
    if g != 1:
        raise ValueError(f"{a} n'est pas inversible modulo {n} (pgcd = {g})")
    return u % n


# ---------------------------------------------------------------------------
# 2.1 Test de primalite de Miller-Rabin (probabiliste)
# ---------------------------------------------------------------------------
def miller_rabin(n, nb_temoins=20):
    """
    Test probabiliste. Principe : si n est premier impair, alors pour tout a
    dans [2, n-2] on a a^(n-1) = 1 mod n (petit theoreme de Fermat), et en
    ecrivant n-1 = 2^s * d (d impair), la suite a^d, a^(2d), ... atteint 1
    uniquement via la valeur -1 (les seules racines carrees de 1 modulo un
    premier sont +/-1). Si une seule de ces conditions echoue pour un temoin a,
    n est COMPOSE avec certitude. Si elle tient pour k temoins aleatoires, n
    est premier avec une probabilite d'erreur <= 4^-k.
    """
    if n < 2:
        return False
    # Petits premiers : elimination rapide des cas evidents.
    for petit in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n == petit:
            return True
        if n % petit == 0:
            return False
    # Ecriture n - 1 = 2^s * d avec d impair.
    d = n - 1
    s = 0
    while d % 2 == 0:
        d //= 2
        s += 1
    # Boucle sur les temoins aleatoires.
    for _ in range(nb_temoins):
        a = 2 + secrets.randbelow(n - 3)      # temoin dans [2, n-2]
        x = pow_mod(a, d, n)
        if x == 1 or x == n - 1:
            continue                           # a ne revele rien, on passe
        compose = True
        for _ in range(s - 1):
            x = (x * x) % n
            if x == n - 1:
                compose = False                # on a trouve -1 : OK pour ce a
                break
        if compose:
            return False                       # a est un temoin de compositude
    return True


# ---------------------------------------------------------------------------
# 2.1 Generation / verification de la cle
# ---------------------------------------------------------------------------
def generer_cle(p=P, q=Q, e=E):
    """Renvoie (N, phi, e, d) a partir de p, q premiers."""
    assert miller_rabin(p), "p n'est pas premier"
    assert miller_rabin(q), "q n'est pas premier"
    N = p * q
    phi = (p - 1) * (q - 1)
    g, _, _ = euclide_etendu(e, phi)
```

```
    assert g == 1, "e et phi(N) ne sont pas premiers entre eux"
    d = inverse_mod(e, phi)
    return N, phi, e, d


# ---------------------------------------------------------------------------
# 2.2 Chiffrement / dechiffrement RSA textbook
# ---------------------------------------------------------------------------
def rsa_enc(m, e, N):
    return pow_mod(m, e, N)


def rsa_dec(c, d, N):
    return pow_mod(c, d, N)


# ===========================================================================
# Demonstrations
# ===========================================================================
def demo_2_1():
    print("=" * 70)
    print("2.1  Generation de la paire de cles")
    print("=" * 70)
    print(f"  p = {P}  premier (Miller-Rabin) ? {miller_rabin(P)}")
    print(f"  q = {Q}  premier (Miller-Rabin) ? {miller_rabin(Q)}")
    N, phi, e, d = generer_cle()
    print(f"  N = p*q = {N}")
    print(f"  phi(N) = (p-1)(q-1) = {phi}")
    print(f"  e = {e}  ;  pgcd(e, phi) = {euclide_etendu(e, phi)[0]}")
    print(f"  d = e^-1 mod phi(N) = {d}")
    print(f"      [Verification 2.1 attendue] 42716440715327873")
    # Controle : e*d = 1 mod phi
    print(f"  controle e*d mod phi = {(e * d) % phi} (doit valoir 1)")
    print()
    return N, phi, e, d


def demo_2_2(N, e, d):
    print("=" * 70)
    print("2.2  Chiffrement et dechiffrement")
    print("=" * 70)
    m = 1394500914838255
    c = rsa_enc(m, e, N)
    print(f"  m = {m}")
    print(f"  c = m^e mod N = {c}")
    print(f"      [Verification 2.2 attendue] 31963660340929498")
    print(f"  controle Dec(Enc(m)) = {rsa_dec(c, d, N)} (doit valoir m : "
          f"{rsa_dec(c, d, N) == m})")
    c_prime = 3632978084399643
    m_prime = rsa_dec(c_prime, d, N)
    print(f"  c' (impose) = {c_prime}")
    print(f"  m' = Dec(c') = c'^d mod N = {m_prime}")
    print(f"      [Verification 2.2 attendue] 45361714772588724")
    print()
    return m, c


def demo_2_3(N, e, d):
    print("=" * 70)
    print("2.3  Cas limites et determinisme")
    print("=" * 70)
    for m in (0, 1, N - 1):
        c = rsa_enc(m, e, N)
        etiquette = {0: "Enc(0)", 1: "Enc(1)", N - 1: "Enc(N-1)"}[m]
        print(f"  {etiquette:9s} = {c}")
```

```
    print("  --> Enc(0)=0, Enc(1)=1, Enc(N-1)=N-1 : ces clairs sont points")
    print("      fixes de l'exponentiation. Un attaquant RECONNAIT le clair")
    print("      directement, sans aucune cryptanalyse.")
    print("  --> RSA textbook est DETERMINISTE : m identique => c identique.")
    print("      Deux cryptogrammes egaux trahissent deux clairs egaux.")
    print()


def demo_2_4(N, e, d, m, c):
    print("=" * 70)
    print("2.4  Malleabilite (homomorphisme multiplicatif)")
    print("=" * 70)
    # c_mal = c * 2^e mod N = Enc(m) * Enc(2) = Enc(2m mod N), sans connaitre m.
    c_mal = (c * pow_mod(2, e, N)) % N
    print(f"  c_mal = c * 2^e mod N = {c_mal}")
    print(f"      [Verification 2.4 attendue] 35103694930280065")
    dec_cmal = rsa_dec(c_mal, d, N)
    print(f"  Dec(c_mal) = {dec_cmal}")
    print(f"      [Verification 2.4 attendue] 2789001829676510")
    print(f"  2m mod N   = {(2 * m) % N}  (egal a Dec(c_mal) : "
          f"{dec_cmal == (2 * m) % N})")
    print("  --> En multipliant le chiffre par 2^e, on obtient le chiffre de 2m")
    print("      SANS connaitre m ni d : RSA textbook est malleable.")
    print()


def demo_2_5(N, e, c):
    print("=" * 70)
    print("2.5  Attaque par dictionnaire (petit espace de messages)")
    print("=" * 70)
    # Scenario : l'attaquant sait que m est un "montant" plausible dans un petit
    # ensemble. On construit >= 1000 candidats incluant la vraie valeur m.
    m_vrai = 1394500914838255
    # On fabrique 1000 candidats "plausibles" autour de la vraie valeur.
    candidats = [m_vrai + delta for delta in range(-499, 501)]  # 1000 valeurs
    print(f"  Taille du dictionnaire : {len(candidats)} candidats")
    print(f"  (il inclut la vraie valeur m = {m_vrai})")
    essais = 0
    trouve = None
    for cand in candidats:
        essais += 1
        if rsa_enc(cand, e, N) == c:      # on chiffre et on compare au c intercepte
            trouve = cand
            break
    print(f"  Cryptogramme intercepte c = {c}")
    print(f"  --> m retrouve = {trouve}  en {essais} essais")
    print(f"      (correct : {trouve == m_vrai})")
    print("  Comme RSA textbook est deterministe et sans aleatoire, chiffrer")
    print("  chaque candidat et comparer a c suffit a identifier le bon clair.")
    print()


if __name__ == "__main__":
    N, phi, e, d = demo_2_1()
    m, c = demo_2_2(N, e, d)
    demo_2_3(N, e, d)
    demo_2_4(N, e, d, m, c)
    demo_2_5(N, e, c)
```

```
======================================================================
2.1  Generation de la paire de cles
======================================================================
  p = 209287069  premier (Miller-Rabin) ? True
  q = 221536921  premier (Miller-Rabin) ? True
  N = p*q = 46364812871374549
  phi(N) = (p-1)(q-1) = 46364812440550560
```

```
  e = 65537  ;  pgcd(e, phi) = 1
  d = e^-1 mod phi(N) = 42716440715327873
      [Verification 2.1 attendue] 42716440715327873
  controle e*d mod phi = 1 (doit valoir 1)

======================================================================
2.2  Chiffrement et dechiffrement
======================================================================
  m = 1394500914838255
  c = m^e mod N = 31963660340929498
      [Verification 2.2 attendue] 31963660340929498
  controle Dec(Enc(m)) = 1394500914838255 (doit valoir m : True)
  c' (impose) = 3632978084399643
  m' = Dec(c') = c'^d mod N = 45361714772588724
      [Verification 2.2 attendue] 45361714772588724

======================================================================
2.3  Cas limites et determinisme
======================================================================
  Enc(0)    = 0
  Enc(1)    = 1
  Enc(N-1)  = 46364812871374548
  --> Enc(0)=0, Enc(1)=1, Enc(N-1)=N-1 : ces clairs sont points
      fixes de l'exponentiation. Un attaquant RECONNAIT le clair
      directement, sans aucune cryptanalyse.
  --> RSA textbook est DETERMINISTE : m identique => c identique.
      Deux cryptogrammes egaux trahissent deux clairs egaux.

======================================================================
2.4  Malleabilite (homomorphisme multiplicatif)
======================================================================
  c_mal = c * 2^e mod N = 35103694930280065
      [Verification 2.4 attendue] 35103694930280065
  Dec(c_mal) = 2789001829676510
      [Verification 2.4 attendue] 2789001829676510
  2m mod N   = 2789001829676510  (egal a Dec(c_mal) : True)
  --> En multipliant le chiffre par 2^e, on obtient le chiffre de 2m
      SANS connaitre m ni d : RSA textbook est malleable.

======================================================================
2.5  Attaque par dictionnaire (petit espace de messages)
======================================================================
  Taille du dictionnaire : 1000 candidats
  (il inclut la vraie valeur m = 1394500914838255)
  Cryptogramme intercepte c = 31963660340929498
  --> m retrouve = 1394500914838255  en 500 essais
      (correct : True)
  Comme RSA textbook est deterministe et sans aleatoire, chiffrer
  chaque candidat et comparer a c suffit a identifier le bon clair.
```

3 Partie 3 – Signatures RSA « textbook »

Même clé (N, e, d) qu'en partie 2. $\text{Sign}(m) = m^d \bmod N$; $\text{Verify}(m, s)$ accepte si $s^e \bmod N = m$.

3.1 3.1 – Implémentation

On signe deux messages, la vérification accepte ; en modifiant un seul bit du message, la vérification **rejette** (voir trace).

3.2 3.2 – Oracle de signature simulé

Un `OracleSignature` détient d « en coulisses » et renvoie $\text{Sign}(m)$ pour tout m soumis, sans révéler d ; on journalise les requêtes. s_1, s_2 correspondent à `[Verification 3.2]`.

3.3 3.3 – Forgerie existentielle

Sans d ni oracle : on choisit s au hasard et on pose $m = s^e \bmod N$; alors Verify(m, s) est *toujours* vraie par construction, car $s \mapsto s^e$ est public. **Peu dangereuse** car l'attaquant ne contrôle pas m (entier quasi aléatoire) : dès que le protocole exige des messages *structurés* ou *hachés*, un $m = s^e$ aléatoire n'a presque jamais de sens. D'où la signature de $H(m)$ plutôt que m.

3.4 3.4 – Forgerie par message choisi (multiplicative)

Cible $m_{\text{cible}} = m_1 m_2 \bmod N$. À partir de s_1, s_2 obtenus de l'oracle, on forge $s_{\text{cible}} = s_1 s_2 \bmod N$ **sans jamais soumettre m_{cible}**. Preuve :

$$s_{\text{cible}}^e = (s_1 s_2)^e = s_1^e s_2^e = m_1 m_2 = m_{\text{cible}} \pmod N,$$

donc Verify$(m_{\text{cible}}, s_{\text{cible}}) = 1$ (valeurs conformes à `[Verification 3.4]`, et m_{cible} absent du journal de l'oracle). **Scénario** : un ordre bancaire dont le montant est le produit de deux montants anodins déjà signés ; l'attaquant combine leurs signatures pour un ordre jamais approuvé. **Parade standard** : ne jamais signer m brut mais $H(m)$ via un schéma à padding probabiliste (**RSA-PSS**, ou un *full-domain hash*), car H détruit la relation multiplicative.

Listing 4 – partie3_signatures.py

```
#!/usr/bin/env python3
# -*- coding: utf-8 -*-
"""
IFT814 - Devoir 2 - Equipe 05
Partie 3 : Signatures RSA "textbook" (meme cle (N, e, d) qu'en Partie 2)

  Sign(sk, m)   = m^d mod N
  Verify(pk, m, s) accepte si  s^e mod N == m

Couvre :
  3.1 Implementation de Sign / Verify + test (dont rejet si le message change)
  3.2 Oracle de signature simule + signatures imposees de la fiche
  3.3 Forgerie existentielle (sans oracle, sans d)
  3.4 Forgerie par message choisi (forgerie multiplicative) + preuve

Primitives implementees a la main, aleatoire via `secrets`.
"""

import secrets

# ---------------------------------------------------------------------------
# Cle de la Partie 2 (recalculee ici pour que le fichier soit autonome)
# ---------------------------------------------------------------------------
P = 209287069
Q = 221536921
E = 65537


def pow_mod(a, b, n):
    r, base, e = 1, a % n, b
    while e > 0:
        if e & 1:
            r = (r * base) % n
        base = (base * base) % n
        e >>= 1
    return r


def euclide_etendu(a, b):
    u_prec, u = 1, 0
```

```
    v_prec, v = 0, 1
    r_prec, r = a, b
    while r != 0:
        q = r_prec // r
        r_prec, r = r, r_prec - q * r
        u_prec, u = u, u_prec - q * u
        v_prec, v = v, v_prec - q * v
    return r_prec, u_prec, v_prec


def inverse_mod(a, n):
    g, u, _ = euclide_etendu(a % n, n)
    if g != 1:
        raise ValueError("non inversible")
    return u % n


N = P * Q
PHI = (P - 1) * (Q - 1)
D = inverse_mod(E, PHI)


# ------------------------------------------------------------------------
# 3.1 Signature et verification
# ------------------------------------------------------------------------
def sign(m, d=D, N=N):
    """Signature RSA textbook : s = m^d mod N (utilise la cle privee d)."""
    return pow_mod(m, d, N)


def verify(m, s, e=E, N=N):
    """Verification : accepte si s^e mod N == m (n'utilise que la cle publique)."""
    return pow_mod(s, e, N) == (m % N)


# ------------------------------------------------------------------------
# 3.2 Oracle de signature simule
# ------------------------------------------------------------------------
class OracleSignature:
    """
    Simule un signataire legitime. L'attaquant peut lui soumettre un message
    et recevoir la signature, mais n'apprend jamais d. On journalise les
    requetes pour pouvoir verifier qu'on n'a PAS interroge l'oracle sur la
    cible (contrainte de la question 3.4).
    """
    def __init__(self, d, N):
        self._d = d          # cle privee, "en coulisses", invisible pour l'attaquant
        self._N = N
        self.journal = []    # historique des messages soumis

    def demander_signature(self, m):
        self.journal.append(m)
        return pow_mod(m, self._d, self._N)


# ========================================================================
# Demonstrations
# ========================================================================
def demo_3_1():
    print("=" * 70)
    print("3.1  Implementation de Sign / Verify")
    print("=" * 70)
    for m in (123456789, 987654321987654):
        s = sign(m)
        print(f"  m = {m}")
```

```
        print(f"    s = Sign(m) = {s}")
        print(f"    Verify(m, s)        = {verify(m, s)}  (doit etre True)")
        # On modifie le message d'un seul bit (XOR 1 sur le bit de poids faible).
        m_altere = m ^ 1
        print(f"    Verify(m^1, s)      = {verify(m_altere, s)}  (doit etre False)")
    print()


def demo_3_2():
    print("=" * 70)
    print("3.2  Oracle de signature simule + signatures imposees")
    print("=" * 70)
    oracle = OracleSignature(D, N)
    m1 = 23151899524468793
    m2 = 15034505925449428
    s1 = oracle.demander_signature(m1)
    s2 = oracle.demander_signature(m2)
    print(f"  m1 = {m1}")
    print(f"  s1 = Sign(m1) = {s1}")
    print(f"      [Verification 3.2 attendue] 40495289341296419")
    print(f"      Verify(m1, s1) = {verify(m1, s1)}")
    print(f"  m2 = {m2}")
    print(f"  s2 = Sign(m2) = {s2}")
    print(f"      [Verification 3.2 attendue] 4958833821949512")
    print(f"      Verify(m2, s2) = {verify(m2, s2)}")
    print()
    return oracle, m1, m2, s1, s2


def demo_3_3():
    print("=" * 70)
    print("3.3  Forgerie existentielle (sans oracle, sans d)")
    print("=" * 70)
    # On choisit d'abord une signature s au hasard, PUIS on en deduit le message.
    s = 2 + secrets.randbelow(N - 3)
    m = pow_mod(s, E, N)          # m = s^e mod N  =>  Verify(m, s) vrai par construction
    print(f"  signature choisie au hasard s = {s}")
    print(f"  message deduit m = s^e mod N  = {m}")
    print(f"  Verify(m, s) = {verify(m, s)}  (toujours True)")
    print("  --> La paire (m, s) est valide, mais m n'est pas choisi par")
    print("      l'attaquant : c'est un entier quasi aleatoire. L'attaque est")
    print("      donc peu dangereuse si le protocole n'accepte que des messages")
    print("      STRUCTURES (format, hachage), ce que l'attaquant ne controle pas.")
    print()


def demo_3_4(oracle, m1, m2, s1, s2):
    print("=" * 70)
    print("3.4  Forgerie par message choisi (forgerie multiplicative)")
    print("=" * 70)
    # Cible : m_cible = m1 * m2 mod N. On NE demande PAS sa signature a l'oracle.
    m_cible = (m1 * m2) % N
    s_cible = (s1 * s2) % N       # s_cible = s1 * s2 mod N
    print(f"  m_cible = m1*m2 mod N = {m_cible}")
    print(f"      [Verification 3.4 attendue] 35566973915918092")
    print(f"  s_cible = s1*s2 mod N = {s_cible}")
    print(f"      [Verification 3.4 attendue] 7736092144531737")
    print(f"  Verify(m_cible, s_cible) = {verify(m_cible, s_cible)}  (doit etre True)")
    # Preuve qu'on n'a jamais soumis m_cible a l'oracle.
    print(f"  m_cible a-t-il ete soumis a l'oracle ? "
          f"{m_cible in oracle.journal}  (doit etre False)")
    print("\n  Preuve algebrique :")
    print("    s_cible^e = (s1*s2)^e = s1^e * s2^e  (mod N)")
    print("              = m1 * m2                  (car Verify(s1)=m1, Verify(s2)=m2)")
    print("              = m_cible                  (mod N)")
```

```
    print("    donc Verify(m_cible, s_cible) = 1, sans jamais signer m_cible.")
    print()


if __name__ == "__main__":
    print(f"N = {N}\ne = {E}\n")
    demo_3_1()
    oracle, m1, m2, s1, s2 = demo_3_2()
    demo_3_3()
    demo_3_4(oracle, m1, m2, s1, s2)
```

```
N = 46364812871374549
e = 65537

======================================================================
3.1  Implementation de Sign / Verify
======================================================================
  m = 123456789
    s = Sign(m) = 15487494352408101
    Verify(m, s)        = True  (doit etre True)
    Verify(m^1, s)      = False  (doit etre False)
  m = 987654321987654
    s = Sign(m) = 29275969331924443
    Verify(m, s)        = True  (doit etre True)
    Verify(m^1, s)      = False  (doit etre False)

======================================================================
3.2  Oracle de signature simule + signatures imposees
======================================================================
  m1 = 23151899524468793
  s1 = Sign(m1) = 40495289341296419
      [Verification 3.2 attendue] 40495289341296419
      Verify(m1, s1) = True
  m2 = 15034505925449428
  s2 = Sign(m2) = 4958833821949512
      [Verification 3.2 attendue] 4958833821949512
      Verify(m2, s2) = True

======================================================================
3.3  Forgerie existentielle (sans oracle, sans d)
======================================================================
  signature choisie au hasard s = 14749202586254523
  message deduit m = s^e mod N  = 18419238530073077
  Verify(m, s) = True  (toujours True)
  --> La paire (m, s) est valide, mais m n'est pas choisi par
      l'attaquant : c'est un entier quasi aleatoire. L'attaque est
      donc peu dangereuse si le protocole n'accepte que des messages
      STRUCTURES (format, hachage), ce que l'attaquant ne controle pas.

======================================================================
3.4  Forgerie par message choisi (forgerie multiplicative)
======================================================================
  m_cible = m1*m2 mod N = 35566973915918092
      [Verification 3.4 attendue] 35566973915918092
  s_cible = s1*s2 mod N = 7736092144531737
      [Verification 3.4 attendue] 7736092144531737
  Verify(m_cible, s_cible) = True  (doit etre True)
  m_cible a-t-il ete soumis a l'oracle ? False  (doit etre False)

  Preuve algebrique :
    s_cible^e = (s1*s2)^e = s1^e * s2^e  (mod N)
              = m1 * m2                  (car Verify(s1)=m1, Verify(s2)=m2)
              = m_cible                  (mod N)
    donc Verify(m_cible, s_cible) = 1, sans jamais signer m_cible.
```

4 Discussion : limites et améliorations

Nos implémentations sont volontairement « textbook » et à petits paramètres (pédagogiques). Limites et pistes : (i) p de 61 bits et RSA de $\approx$56 bits sont cassables – en production, $p \geq 3072$ bits

et $N \geq 2048$ bits ; (ii) Diffie-Hellman doit être **authentifié** (signatures / PKI) contre le MITM ; (iii) RSA doit utiliser **OAEP** (chiffrement) et **PSS** (signature) pour écarter déterminisme, malléabilité, dictionnaire et forgeries ; (iv) le chiffrement par flot de 1.3 devrait dériver une clé longue via une **KDF** et ne jamais réutiliser un flot de clé court et périodique.

5 Bibliographie et sources consultées

— Énoncé du Devoir 2, IFT814 – Cryptographie, Automne 2026 (et fiche Équipe 05).
— Nombre de Mersenne premier – Wikipédia.
— Test de primalité de Miller-Rabin – Wikipédia.
— Boutisme (big-endian) – Wikipédia.
— Notes de cours IFT814 (Diffie-Hellman, RSA, OAEP, PSS, logarithme discret).